Lab 7: Fuzzing for Input Validation Bugs

Software Engineering · PES University, Dept. of CSE · **PES1UG24CS569**
Assigned repository: L6_L7_20_food_delivery (Food Delivery) · Submission: github.com/Sonuaadi0706/SE-Labs-PES1UG24CS569 (folder `Lab7/`)

Item	Value
Application under test	`food_delivery.py` — `parse_order`, `delivery_eta`, `coupon_discount` (the handout's `processor.py`)
Fuzz test file	`test_food_delivery.py` (the handout's `test_processor.py`) — 11 Hypothesis tests, 500 generated inputs each
Environment	macOS · Python 3.15.0 · Hypothesis 6.170.15 · pytest 9.2.0
Result	Buggy module: **8 failed**, 3 passed · Fixed module: **11 passed**

Step 1 — Install dependencies

`pip install hypothesis` `pip install pytest` (installed into a virtual environment; versions above).

Step 2 — Run the buggy application

`python food_delivery.py`



Figure 1 — Buggy program with the example inputs: works.

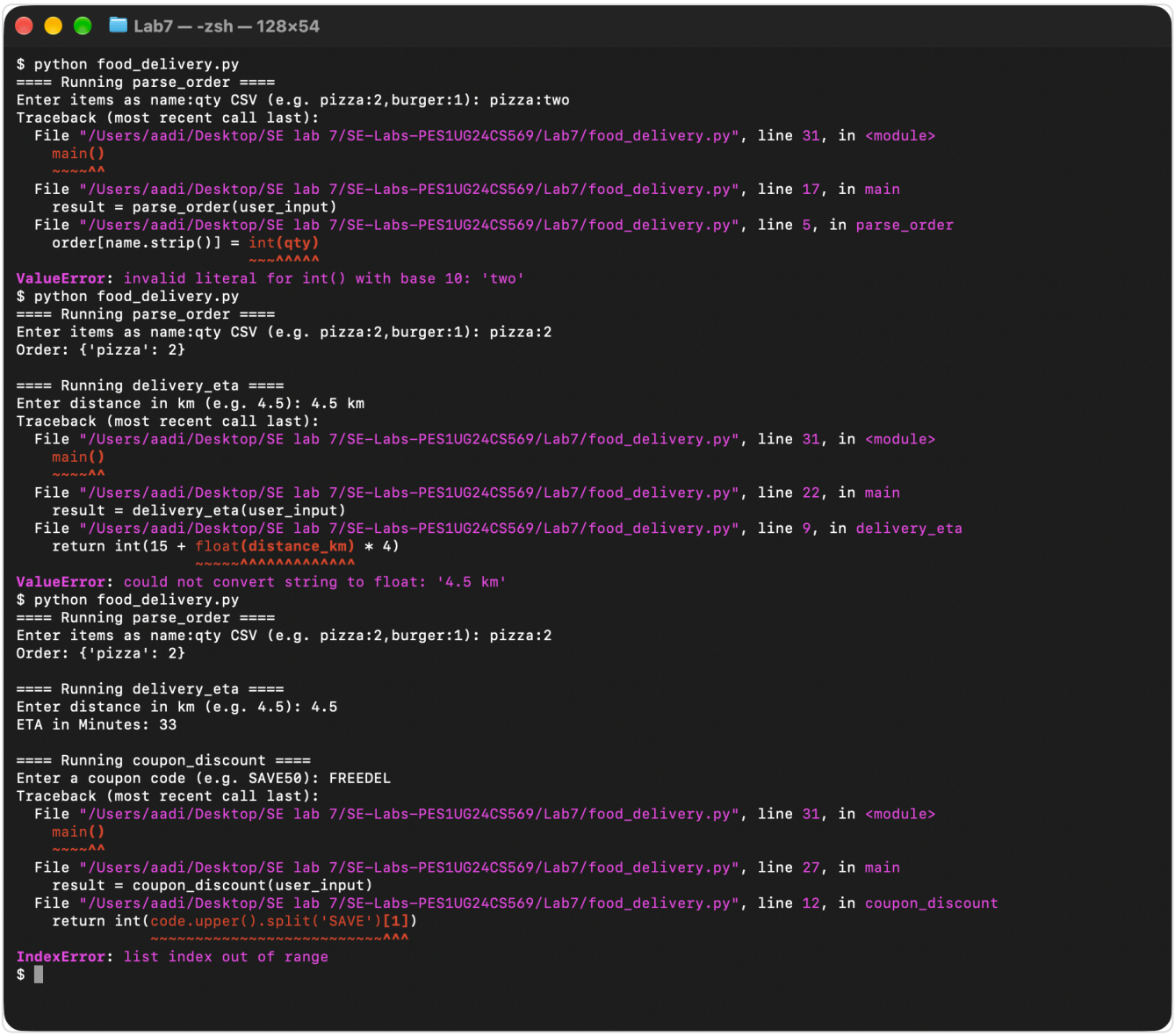


Figure 2 — Buggy program with one bad value per function: `pizza:two` → ValueError, `4.5 km` → ValueError, `FREEDEL` → IndexError. Each crash ends the whole program.

Step 3 — Complete the fuzz tests with Hypothesis

The three template tests (`@given(st.text() | st.none())`, fail if the function raises anything) were completed, and eight property tests were added so the fuzzer also catches *wrong answers*, not just crashes:

Function	Tests	Properties checked
`parse_order`	no_crash, returns_valid_order, round_trip, merges_duplicate_items	never raises; result is a dict of non-empty names → int 1–99; valid orders parse back exactly; repeated items are added
`delivery_eta`	no_crash, result_in_range, correct_for_valid_distance, never_shorter_for_longer_trip	never raises; result is `None` or int 15–215; equals `ceil(15 + 4·d)`; longer trip never gets a shorter ETA
`coupon_discount`	no_crash, result_in_range, valid_codes	never raises; result is int 0–100; `SAVE<n>` in any case returns `n`

Inputs come from `st.text()`, `st.none()`, integers, floats (incl. NaN/∞), booleans and lists, plus "order-like", "number-like" and "coupon-like" text built from the characters each parser cares about, so the fuzzer reaches deeper branches. Known bad inputs are pinned with `@example(...)` as regression tests.

Step 4 — Run the fuzz tests on the buggy application

```
python -m pytest test_food_delivery.py
```

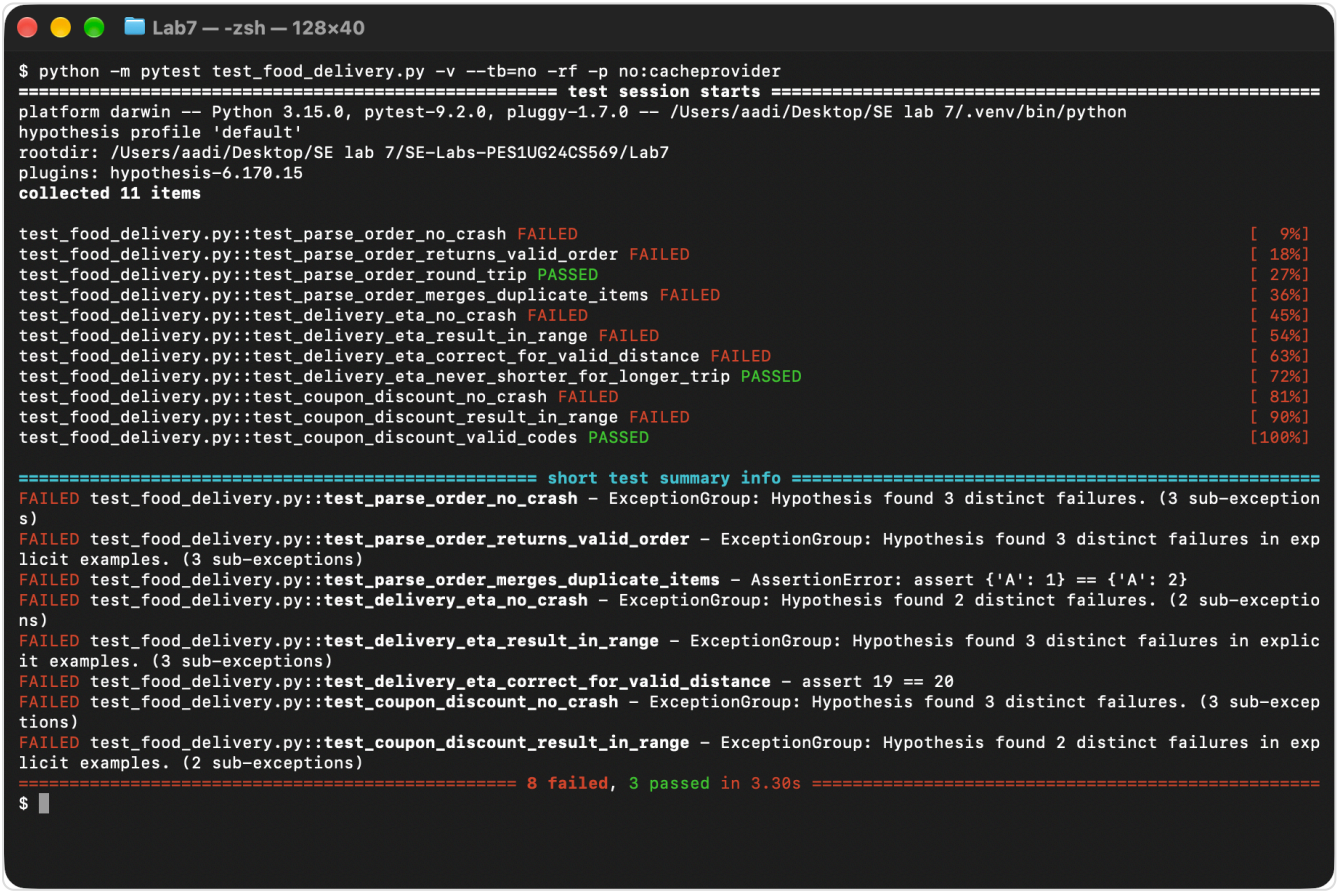


Figure 3 — 8 of 11 fuzz tests fail on the buggy module.

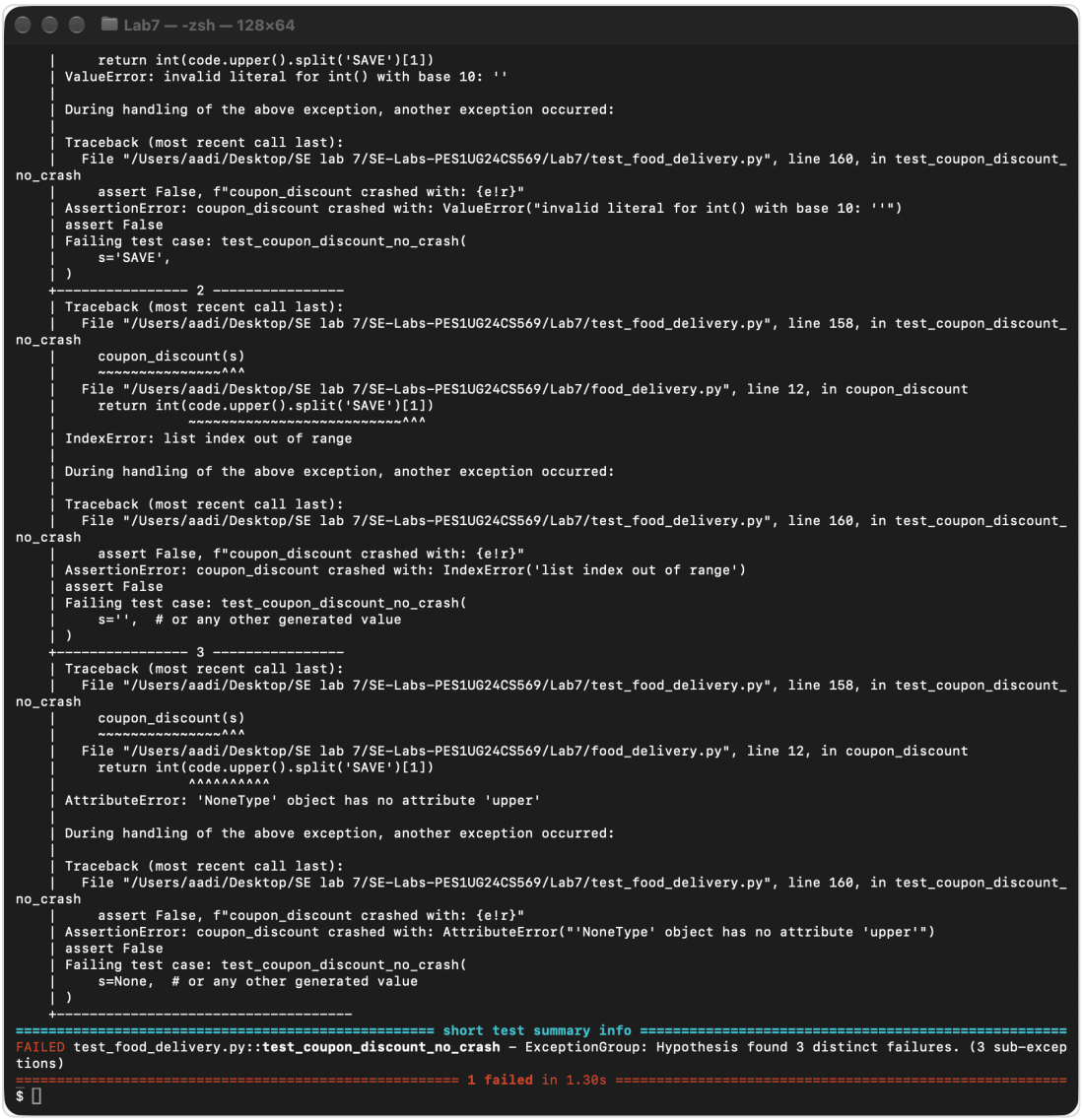


Figure 4 — Detail for `coupon_discount`: Hypothesis found and shrank three distinct crashes, from `'SAVE'`, `''` and `None`.

Bugs found

#	Function	Failing input	Buggy behaviour
1	parse_order	`None`	AttributeError: 'NoneType' has no attribute 'split'
2	parse_order	`''`, `'pizza:2,'`	ValueError: not enough values to unpack
3	parse_order	`':'`, `'pizza:two'`, `'a:1:2'`	ValueError (empty / non-numeric quantity, too many ':')
4	parse_order	`'pizza:-3'`	negative quantity accepted
5	parse_order	`'pizza:' + '9'*5000`	ValueError: exceeds the 4300-digit int limit
6	parse_order	`'A:1,A:1'`	repeated item overwritten: {'A': 1} instead of {'A': 2}
7	delivery_eta	`None`	TypeError from float()
8	delivery_eta	`''`, `'4.5 km'`	ValueError: could not convert string to float
9	delivery_eta	`'nan'`, `'inf'`	ValueError / OverflowError converting to int
10	delivery_eta	`'-5'`, `'1e3'`	impossible ETAs: –5 min, 4015 min
11	delivery_eta	`1.125`	19.5 min truncated to 19 (now rounded up to 20)
12	coupon_discount	`None`	AttributeError: 'NoneType' has no attribute 'upper'
13	coupon_discount	`''`, `'FREEDEL'`	IndexError: list index out of range
14	coupon_discount	`'SAVE'`	ValueError: invalid literal for int()
15	coupon_discount	`'SAVE-50'`, `'SAVE99999'`, `'XSAVE50'`	negative, unlimited or prefixed discounts accepted

Step 5 — Fix the application

No function raises any more. Invalid input returns a clear "invalid" value, and `main()` prints a helpful message instead of a traceback.

Function	Invalid input returns	Validation added
`parse_order`	`{}`	must be a str; blank items skipped; exactly one ':' per item; name non-empty and printable; quantity is 1–2 ASCII digits (regex `fullmatch`); repeated items added; total 1–99
`delivery_eta`	`None`	accepts numbers (not bool) or a plain decimal string; regex rejects units, signs, nan, inf, 1e3; must be finite and in (0, 50] km; `math.ceil` instead of truncation
`coupon_discount`	`0`	must be an ASCII str; spaces stripped; whole code must match `SAVE([0-9]{1,3})`; amount 1–100

Checking digits with a regex before calling `int()`/`float()` matters: `int()` also accepts `'-3'`, `'1_0'` and Unicode digits like `'٢'`, and `float()` also accepts `'nan'`, `'inf'` and `'1e3'`.

```
python food_delivery.py
```



Figure 5 — Fixed program: valid input gives the same results as before.

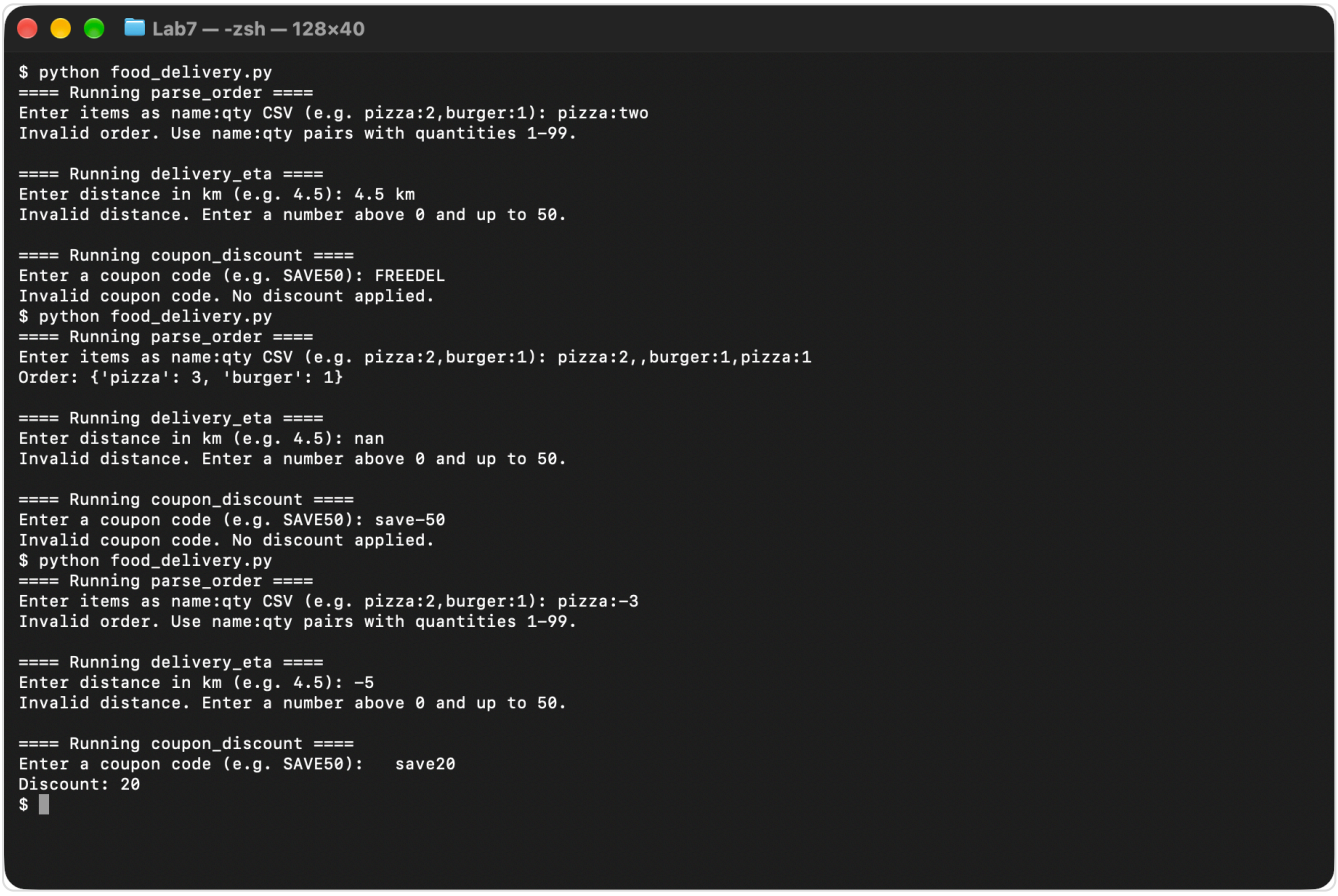


Figure 6 — Fixed program: the inputs that used to crash are rejected with a message; repeated items are added together (pizza 2 + 1 = 3); `' save20 '` is accepted.

Step 6 — Re-run the fuzz tests on the fixed application

```
python -m pytest test_food_delivery.py
```

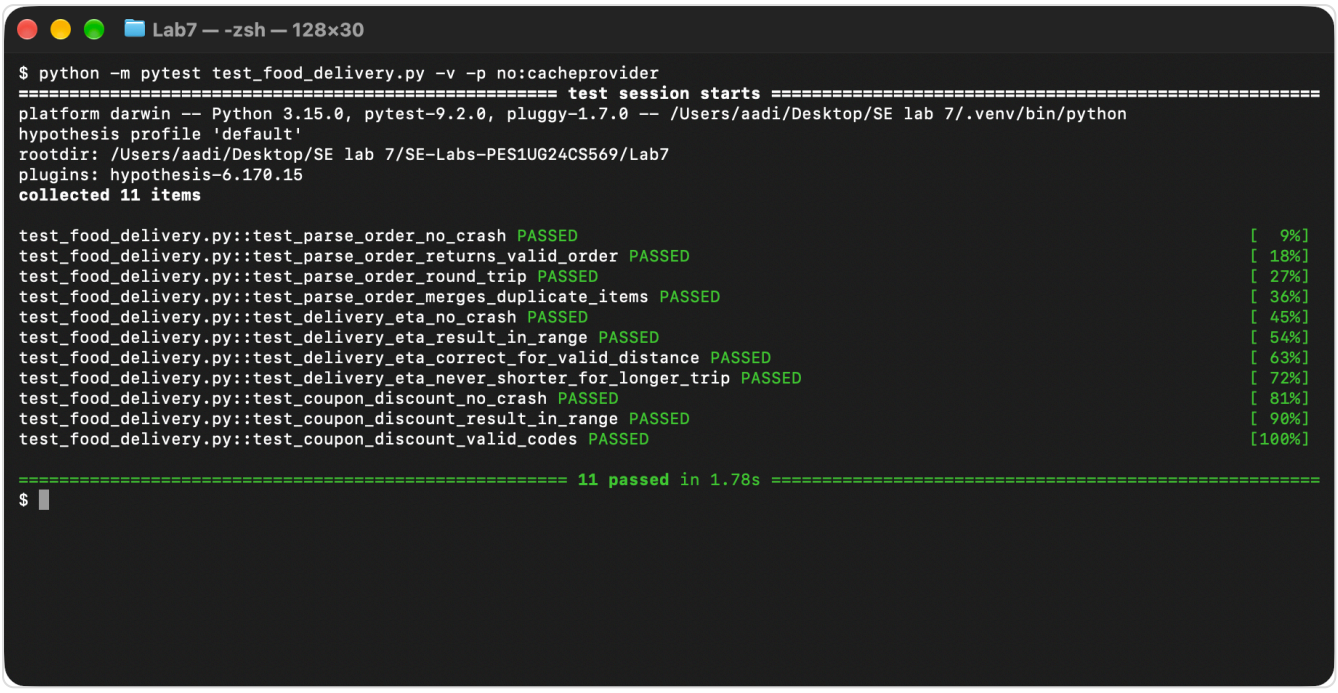


Figure 7 — All 11 fuzz tests pass on the fixed module.

The suite also passed with 6 different random seeds, and a separate stress run of 20,000 random inputs per function found no crash or out-of-range result. While testing the fix, Hypothesis found one more edge case: for a tiny distance such as `5.7e-55` km, `15 + 4·d` rounds to exactly 15.0 in floating point. An ETA of 15 minutes is correct, so the test's lower bound was changed from `> 15` to `>= 15`.

Deliverables

Deliverable	File (in `Lab7/`)
1. PDF document with the screenshots	`Lab7_FuzzTesting_Report.pdf` (this document)
2. Fixed application Python script	`food_delivery.py`
3. Updated test script	`test_food_delivery.py`
Full pytest output before / after the fix	`logs/pytest_before_fix.txt`, `logs/pytest_after_fix.txt`